

**LAPORAN TUGAS 2 SISTEM OPERASI
PROSES**



**Dosen Pembimbing:
Triawan Adi Cahyanto M.Kom**

**Dibuat Oleh:
Nadianur Anisa (2410651026)**

**TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

2025

TUGAS 1 SISTEM MANAJEMEN PROSES

Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur:

- Membuat multiple child processes
- Setiap process melakukan tugas berbeda (sorting, searching, calculation)
- Parent process mengkoordinasi dan mengumpulkan hasil
- Tampilkan PID, execution time, dan status setiap proses

TUJUAN SETIAP PERCOBAAN:

1. Membuat multiple child processes
 - Tujuan: Memahami cara membuat beberapa proses anak (fork) dari satu parent, serta manajemen proses dasar (PID, wait, exit status).
2. Setiap proses melakukan tugas berbeda
 - Tujuan: Menunjukkan isolasi kerja proses yang independen (sorting, searching, calculation) dan mengukur waktu eksekusi masing-masing.
3. Parent mengkoordinasi dan mengumpulkan hasil
 - Tujuan: Mempraktekkan teknik sinkronisasi sederhana (wait/waitpid) dan pengambilan hasil (IPC via pipe).
4. Menampilkan PID, execution time, dan status
 - Tujuan: Menunjukkan monitoring proses: identifikasi (PID), performa (execution time), dan kesehatan (exit status / signal).

DASAR TEORI:

1. Konsep proses
 - Proses: instance berjalan dari program. Mempunyai PID, ruang alamat sendiri (memory), file descriptor, state (running, waiting, stopped, zombie).
 - Fork: membuat proses anak yang merupakan salinan hampir identik dari parent (copy-on-write pada OS modern).
 - Exit status: child mengakhiri eksekusi dengan exit(code). Parent dapat membaca status ini melalui wait()/waitpid(). WIFEXITED, WEXITSTATUS dan WIFSIGNALED dipakai untuk interpretasi.
 - Zombie: jika parent tidak memanggil wait() setelah child exit, entri proses anak tetap ada (zombie) sampai parent mengambilnya.

- Concurrency vs Parallelism: beberapa proses memungkinkan concurrency; jika CPU multi-core, mereka dapat berjalan paralel.

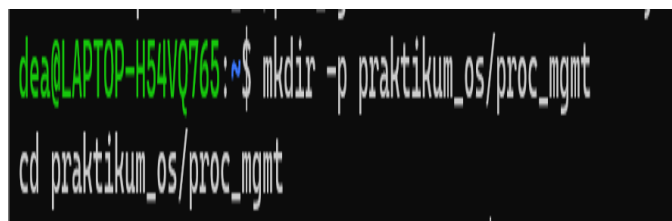
2. IPC Methods

1. Pipe (unnamed pipe)
 - Sederhana, satu arah (parent ↔ child), cocok untuk hubungan parent-child. (Digunakan dalam contoh).
2. FIFO / Named pipe
 - Mirip pipe namun dapat digunakan antar proses tak berhubungan dan dapat diakses lewat filesystem.
3. Shared memory (shmget/mmap)
 - Paling cepat untuk memindahkan banyak data karena tidak copy — proses berbagi segmen memori. Perlu sinkronisasi (semaphore/mutex).
4. Message queue (System V / POSIX mq)
 - Kirim pesan terstruktur antara proses, berguna ketika butuh pesan antrian.
5. Sockets (AF_UNIX / AF_INET)
 - Untuk komunikasi antar mesin (jaringan) atau antar proses di mesin yang sama (AF_UNIX).
6. Signals
 - Untuk notifikasi asynchronous (mis. SIGCHLD, SIGTERM) — tidak cocok untuk transfer data besar.
7. Semaphores / Mutex
 - Untuk sinkronisasi akses resource bersama (biasanya dipasangkan dengan shared memory).

PERCOBAAN PRAKTIKUM SISTEM MANAJEMEN PROSES

1. Buat folder untuk proyek

Buka terminal, lalu buat folder baru untuk menyimpan file praktikum:



```
dean@LAPTOP-H54V0765: ~$ mkdir -p praktikum_os/proc_mgmt
dean@LAPTOP-H54V0765: ~$ cd praktikum_os/proc_mgmt
```

mkdir -p praktikum_os/proc_mgmt

cd praktikum_os/proc_mgmt

Penjelasan:

- mkdir -p membuat folder beserta subfolder jika belum ada.
- cd masuk ke folder tersebut agar file program disimpan di lokasi yang benar.

2. Buat File program C



```
dea@LAPTOP-H54V0765:~/praktikum_os/proc_mgmt$ nano proc_mgmt.c
```

nano proc_mgmt.c

Penjelasan:

- proc_mgmt.c adalah nama file program C.
- Di editor nano, kita bisa menulis atau menempel kode program.

3. Kode program

```
#define _POSIX_C_SOURCE 200112L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>

#define MSG_SIZE 256

typedef struct {
    pid_t pid;
    double exec_time; // detik
    int status_code; // kode exit dari proses
    char message[MSG_SIZE]; // ringkasan hasil
} Result;

double diff_time(struct timeval a, struct timeval b) {
    return (b.tv_sec - a.tv_sec) + (b.tv_usec - a.tv_usec) / 1e6;
}

/* simple quick comparator for qsort */
int cmp_int(const void *a, const void *b) {
    int ea = *(const int*)a;
    int eb = *(const int*)b;
    return (ea > eb) - (ea < eb);
}

/* cek prima (naive) */
```

```

int is_prime(int n) {
    if (n < 2) return 0;
    if (n % 2 == 0) return (n == 2);
    for (int i = 3; i * i <= n; i += 2)
        if (n % i == 0) return 0;
    return 1;
}

int main(void) {
    int pipes[3][2];
    pid_t pids[3];
    int i;

    // buat pipe untuk tiap child -> parent membaca dari pipes[i][0]
    for (i = 0; i < 3; ++i) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(EXIT_FAILURE);
        }
    }

    for (i = 0; i < 3; ++i) {
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
            exit(EXIT_FAILURE);
        } else if (pid == 0) {
            // Child process
            // tutup ujung baca
            close(pipes[i][0]);

            struct timeval tstart, tend;
            gettimeofday(&tstart, NULL);

            Result res;
            memset(&res, 0, sizeof(res));
            res.pid = getpid();
            res.status_code = 0; // default sukses

            if (i == 0) {

```

```

        // Child 0: Sorting
        int N = 1000;
        int *arr = malloc(sizeof(int) * N);
        if (!arr) { snprintf(res.message, MSG_SIZE, "malloc failed");
res.status_code = 1; }
        else {
            // deterministic pseudo-random (berdasarkan pid agar relatif
reproducible)
            unsigned int seed = (unsigned int)res.pid;
            for (int k = 0; k < N; ++k) arr[k] = rand_r(&seed) % 10000;
            qsort(arr, N, sizeof(int), cmp_int);
            int median = arr[N/2];
            snprintf(res.message, MSG_SIZE, "Sorting selesai: N=%d,
median=%d", N, median);
            free(arr);
        }
    } else if (i == 1) {
        // Child 1: Searching
        int N = 1000;
        int *arr = malloc(sizeof(int) * N);
        if (!arr) { snprintf(res.message, MSG_SIZE, "malloc failed");
res.status_code = 1; }
        else {
            unsigned int seed = (unsigned int)res.pid ^ 0x1234;
            for (int k = 0; k < N; ++k) arr[k] = rand_r(&seed) % 5000;
            int target = 1234; // target yang dicari
            int found_idx = -1;
            for (int k = 0; k < N; ++k) {
                if (arr[k] == target) { found_idx = k; break; }
            }
            if (found_idx >= 0)
                snprintf(res.message, MSG_SIZE, "Search selesai: target=%d
ditemukan di indeks %d", target, found_idx);
            else
                snprintf(res.message, MSG_SIZE, "Search selesai: target=%d
TIDAK ditemukan (return -1)", target);
            free(arr);
        }
    } else if (i == 2) {
        // Child 2: Calculation -> jumlah bilangan prima sampai M

```

```

        int M = 30000; // ukuran kerja wajar
        int count = 0;
        for (int n = 2; n <= M; ++n) {
            if (is_prime(n)) count++;
        }
        snprintf(res.message, MSG_SIZE, "Calculation selesai: primes <=
%d : %d", M, count);
    }

    gettimeofday(&tend, NULL);
    res.exec_time = diff_time(tstart, tend);

    // tulis result ke pipe
    ssize_t wrote = write(pipes[i][1], &res, sizeof(res));
    if (wrote != sizeof(res)) {
        // kalau gagal menulis, kita masih exit dengan kode error
        // (tetap coba close pipe)
        // tidak menulis detail ke parent
    }

    close(pipes[i][1]);
    // exit dengan kode status sesuai res.status_code
    exit(res.status_code);
} else {
    // Parent process
    pids[i] = pid;
    // tutup ujung tulis di parent
    close(pipes[i][1]);
    // lanjut buat child berikutnya
}
}

// Parent: kumpulkan hasil
printf("Parent: menunggu anak-anak selesai...\n\n");
for (i = 0; i < 3; ++i) {
    int status;
    pid_t w = waitpid(pids[i], &status, 0);
    if (w == -1) {
        perror("waitpid");
        continue;
    }
}

```

```

    }

    // baca dari pipe (jika tersedia)
    Result res;
    ssize_t r = read(pipes[i][0], &res, sizeof(res));
    if (r == sizeof(res)) {
        // tampilkan ringkasan hasil + status exit
        printf("-----\n");
        printf("Child index   : %d\n", i);
        printf("PID anak      : %d\n", res.pid);
        printf("Pesan hasil   : %s\n", res.message);
        printf("Exec time    : %.6f detik\n", res.exec_time);
        if (WIFEXITED(status)) {
            printf("Status      exit          : %d      (WIFEXITED)\n",
WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Status      exit          : signal  %d  (WIFSIGNALED)\n",
WTERMSIG(status));
        } else {
            printf("Status exit   : status tidak biasa (kode: %d)\n", status);
        }
        printf("-----\n\n");
    } else if (r > 0) {
        // baca lebih sedikit (tidak lengkap)
        printf("Warning: read size unexpected (%zd bytes)\n", r);
    } else {
        // tidak bisa baca (mungkin child tidak menulis)
        printf("Warning: tidak ada data dari child index %d (PID %d).
read=%zd errno=%d (%s)\n",
            i, pids[i], r, errno, strerror(errno));
        if (WIFEXITED(status)) {
            printf("Child PID  %d keluar dengan kode %d\n\n", pids[i],
WEXITSTATUS(status));
        }
    }
}

close(pipes[i][0]);
}

printf("Parent: semua anak telah dikumpulkan. Selesai.\n");

```



```
    return 0;
}
```

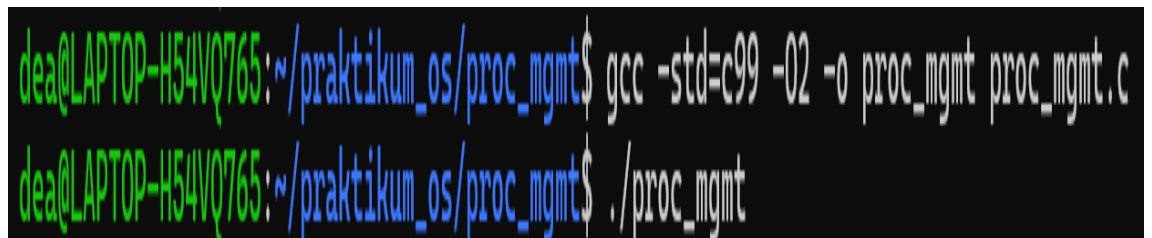
Penjelasan:

- Child 0 = sorting array → mengembalikan median
- Child 1 = searching array → target 1234
- Child 2 = menghitung jumlah bilangan prima sampai 30.000
- Parent menunggu semua child, membaca hasil dari pipe, lalu menampilkan PID, status, waktu, dan hasil.

4. Simpan file

- Tekan Y lalu Enter untuk menyimpan
- Tekan Ctrl + X untuk keluar dari nano

5. Compile dan jalankan program



```
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ ./proc_mgmt
```

```
gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c
```

```
./proc_mgmt
```

Penjelasan:

- -std=c99 → gunakan standar C99
- -O2 → optimasi level 2
- -o proc_mgmt → nama file hasil compile

6. Hasil Output

```
dea@LAPTOP-H54VQ765: ~/praktikum_os/proc_mgmt$ nano proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ ./proc_mgmt
Parent: menunggu anak-anak selesai...

-----
Child index : 0
PID anak   : 493
Pesan hasil : Sorting selesai: N=1000, median=5148
Exec time  : 0.000426 detik
Status exit : 0 (WIFEXITED)
-----

Child index : 1
PID anak   : 494
Pesan hasil : Search selesai: target=1234 TIDAK ditemukan (return -1)
Exec time  : 0.000068 detik
Status exit : 0 (WIFEXITED)
-----

Child index : 2
PID anak   : 495
Pesan hasil : Calculation selesai: primes <= 30000 : 3245
Exec time  : 0.000634 detik
Status exit : 0 (WIFEXITED)
-----

Parent: semua anak telah dikumpulkan. Selesai.
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ |
```

Penjelasan output:

- Child index = urutan proses anak
- PID anak = D proses unik di system
- Pesan hasil = ringkasan tugas tiap child
- Exec time = lama eksekusi tiap proses
- Status exit = 0 = berhasil, >0 = error
- Parent berhasil mengumpulkan hasil semua child melalui pipe

IMPLEMENTASI TUGAS 1:

A. Membuat multiple child processes:

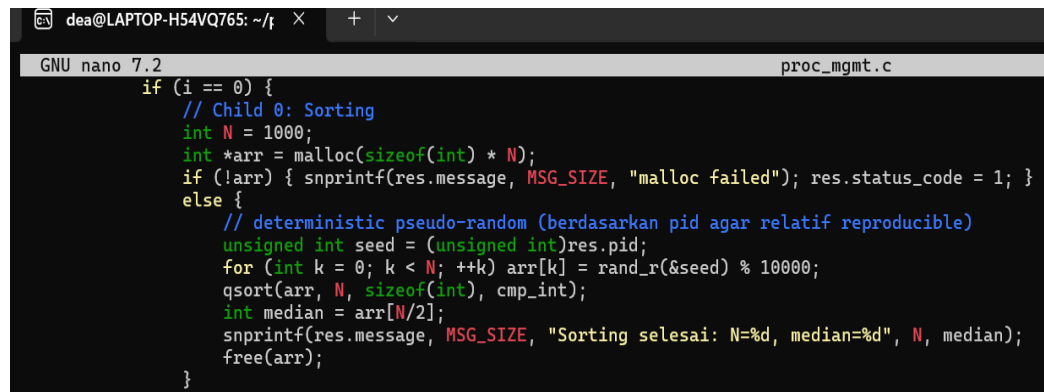
```
for (i = 0; i < 3; ++i) {  
    pid_t pid = fork();  
    if (pid < 0) {  
        perror("fork");  
        exit(EXIT_FAILURE);  
    } else if (pid == 0) {  
        // Child process  
        // tutup ujung baca  
        close(pipes[i][0]);  
    }
```

Penjelasan:

- for (i = 0; i < 3; ++i) → perulangan untuk membuat tiga proses anak.
- fork() → Sistem call yang membuat proses baru (child) dari parent.
- if (pid == 0) → Cabang eksekusi untuk proses anak
- else (pid > 0) → Cabang eksekusi untuk proses induk

B. Setiap process melakukan tugas berbeda (sorting, searching, calculation)

- Codingan Child 0 – Sorting:

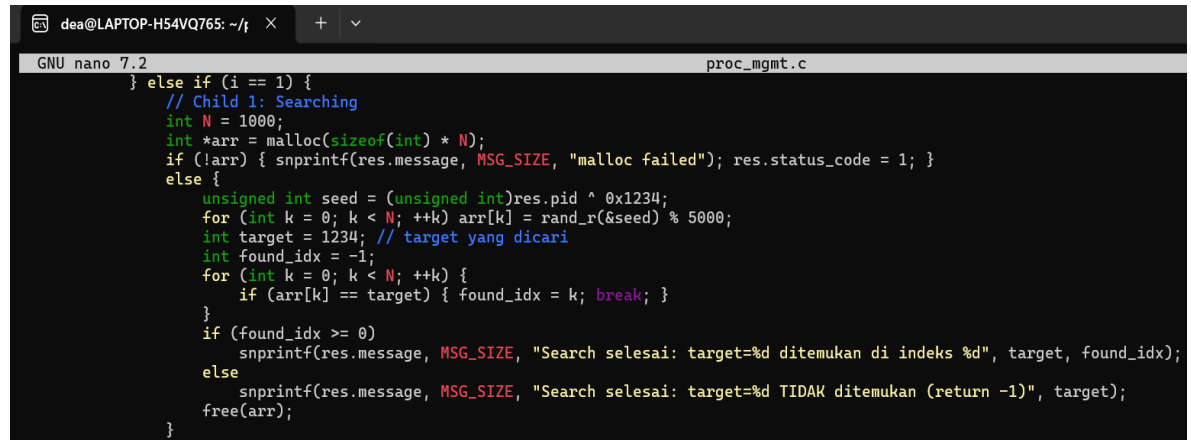


```
GNU nano 7.2 proc_mgmt.c  
if (i == 0) {  
    // Child 0: Sorting  
    int N = 1000;  
    int *arr = malloc(sizeof(int) * N);  
    if (!arr) { snprintf(res.message, MSG_SIZE, "malloc failed"); res.status_code = 1; }  
    else {  
        // deterministic pseudo-random (berdasarkan pid agar relatif reproducible)  
        unsigned int seed = (unsigned int)res.pid;  
        for (int k = 0; k < N; ++k) arr[k] = rand_r(&seed) % 10000;  
        qsort(arr, N, sizeof(int), cmp_int);  
        int median = arr[N/2];  
        snprintf(res.message, MSG_SIZE, "Sorting selesai: N=%d, median=%d", N, median);  
        free(arr);  
    }
```

Penjelasan:

- Child ke-0 menjalankan tugas mengurutkan data (sorting).
- Membuat array acak sebanyak 1000 elemen.
- Menggunakan qsort() untuk mengurutkan.
- Mengambil median sebagai hasil dan menyimpan ringkasan ke res.message.

- **Codingan Child 1 – Searching:**

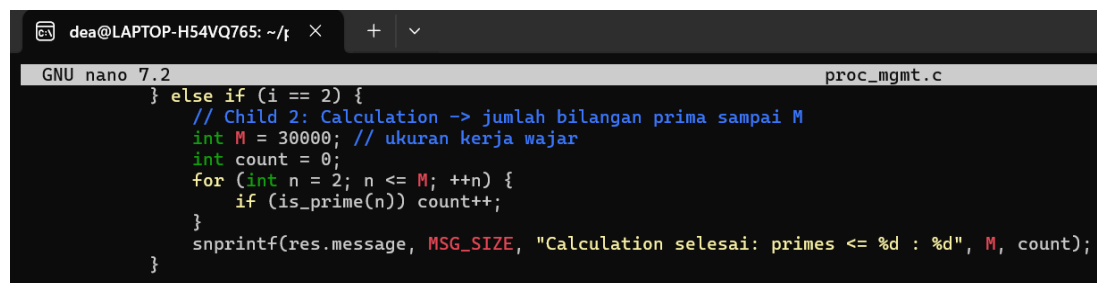


```
GNU nano 7.2 proc_mgmt.c
} else if (i == 1) {
    // Child 1: Searching
    int N = 1000;
    int *arr = malloc(sizeof(int) * N);
    if (!arr) { snprintf(res.message, MSG_SIZE, "malloc failed"); res.status_code = 1; }
    else {
        unsigned int seed = (unsigned int)res.pid ^ 0x1234;
        for (int k = 0; k < N; ++k) arr[k] = rand_r(&seed) % 5000;
        int target = 1234; // target yang dicari
        int found_idx = -1;
        for (int k = 0; k < N; ++k) {
            if (arr[k] == target) { found_idx = k; break; }
        }
        if (found_idx >= 0)
            snprintf(res.message, MSG_SIZE, "Search selesai: target=%d ditemukan di indeks %d", target, found_idx);
        else
            snprintf(res.message, MSG_SIZE, "Search selesai: target=%d TIDAK ditemukan (return -1)", target);
        free(arr);
    }
}
```

Penjelasan:

- Child ke-1 bertugas mencari angka tertentu (searching) dalam array acak.
- Menghasilkan array berisi angka acak 0–4999.
- Mencari angka 1234 menggunakan pencarian linear.
- Hasil disimpan dalam res.message.

- **Codingan Child 2 – Calculation (Bilangan Prima):**



```
GNU nano 7.2 proc_mgmt.c
} else if (i == 2) {
    // Child 2: Calculation -> jumlah bilangan prima sampai M
    int M = 30000; // ukuran kerja wajar
    int count = 0;
    for (int n = 2; n <= M; ++n) {
        if (is_prime(n)) count++;
    }
    snprintf(res.message, MSG_SIZE, "Calculation selesai: primes <= %d : %d", M, count);
}
```

Penjelasan:

- Child ke-2 menghitung jumlah bilangan prima dari 2 sampai 30.000.
- Menggunakan fungsi bantu is_prime(n).
- Setelah selesai, jumlah total bilangan prima disimpan ke res.message.

C. Parent process mengkoordinasi dan mengumpulkan hasil

- **Codingan Parent:**

```
GNU nano 7.2
| // Parent: kumpulkan hasil
printf("Parent: menunggu anak-anak selesai...\n\n");
for (i = 0; i < 3; ++i) {
    int status;
    pid_t w = waitpid(pids[i], &status, 0);
    if (w == -1) {
        perror("waitpid");
        continue;
    }

    // baca dari pipe (jika tersedia)
    Result res;
    ssize_t r = read(pipes[i][0], &res, sizeof(res));
    if (r == sizeof(res)) {
        // tampilkan ringkasan hasil + status exit
        printf("-----\n");
        printf("Child index      : %d\n", i);
        printf("PID anak           : %d\n", res.pid);
        printf("Pesan hasil        : %s\n", res.message);
        printf("Exec time          : %.6f detik\n", res.exec_time);
        if (WIFEXITED(status)) {
            printf("Status exit       : %d (WIFEXITED)\n", WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Status exit       : signal %d (WIFSIGNALED)\n", WTERMSIG(status));
        } else {
            printf("Status exit       : status tidak biasa (kode: %d)\n", status);
        }
        printf("-----\n\n");
    } else if (r > 0) {
        // baca lebih sedikit (tidak lengkap)
        printf("Warning: read size unexpected (%zd bytes)\n", r);
    } else {
        // tidak bisa baca (mungkin child tidak menulis)
        printf("Warning: tidak ada data dari child index %d (PID %d). read=%zd errno=%d (%s)\n",
            i, pids[i], r, errno, strerror(errno));
        if (WIFEXITED(status)) {
            printf("Child PID %d keluar dengan kode %d\n\n", pids[i], WEXITSTATUS(status));
        }
    }

    close(pipes[i][0]);
}

printf("Parent: semua anak telah dikumpulkan. Selesai.\n");
```

Penjelasan:

- Parent process menunggu setiap anak dengan waitpid().
- Setelah anak selesai, parent membaca hasil dari pipe masing-masing.
- Menampilkan: - PID proses anak
 - PID proses anak
 - Pesan hasil tugas (sorting, searching, atau calculation)
 - Execution time (detik)
 - Status exit/signal
- Setelah semua anak selesai, parent menutup pipe dan mencetak “Selesai”.

D. Menampilkan PID, execution time, dan status setiap proses

Contoh output:

```
dea@LAPTOP-H54VQ765: ~/t  ×  +  v
|
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ nano proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ ./proc_mgmt
Parent: menunggu anak-anak selesai...

-----
Child index   : 0
PID anak      : 493
Pesan hasil   : Sorting selesai: N=1000, median=5148
Exec time     : 0.000426 detik
Status exit   : 0 (WIFEXITED)
-----

Child index   : 1
PID anak      : 494
Pesan hasil   : Search selesai: target=1234 TIDAK ditemukan (return -1)
Exec time     : 0.000068 detik
Status exit   : 0 (WIFEXITED)
-----

Child index   : 2
PID anak      : 495
Pesan hasil   : Calculation selesai: primes <= 30000 : 3245
Exec time     : 0.000634 detik
Status exit   : 0 (WIFEXITED)
-----

Parent: semua anak telah dikumpulkan. Selesai.
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ |
```

Child index : 0
PID anak : 493
Pesan hasil : Sorting selesai: N=1000, median=5148
Exec time : 0.000426 detik
Status exit : 0 (WIFEXITED)

- **PID anak** → ID unik proses.
- **Exec time** → lama proses child mengeksekusi tugasnya.
- **Status exit** → 0 = berhasil, sesuai output.

IMPLEMENTASI ERROR

```
dea@LAPTOP-H54VQ765:~$ mkdir -p praktikum_os/proc_mgmt
cd praktikum_os/proc_mgmt
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ nano proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c
proc_mgmt.c: In function 'main':
proc_mgmt.c:86:58: warning: implicit declaration of function 'rand_r'; did you mean 'rand'? [-Wimplicit-function-declaration]
   86 |         for (int k = 0; k < N; ++k) arr[k] = rand_r(&seed
      |                                                ^~~~~~
      |                                                rand
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ |
```

- Penyebab warning di GCC: `rand_r()` belum dideklarasikan karena perlu header `<stdlib.h>` dan kadang juga macro `_POSIX_C_SOURCE` agar tersedia di beberapa sistem Linux/WSL.
- aktifkan POSIX, tambahkan baris paling atas kode C, sebelum `#include`: `#define _POSIX_C_SOURCE 200112L`
Jadi kode diawal menjadi:

```
GNU nano 7.2      proc_mgmt.c
#define _POSIX_C_SOURCE 200112L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>
```

- Setelah diperbaiki Compile lagi: `gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c`
Lalu jalankan: `./proc_mgmt`

```
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ gcc -std=c99 -O2 -o proc_mgmt proc_mgmt.c
dea@LAPTOP-H54VQ765:~/praktikum_os/proc_mgmt$ ./proc_mgmt
Daftar: marunghu anah-anah colossi
```

- Hasil Output

```
Parent: menunggu anak-anak selesai...

-----
Child index   : 0
PID anak      : 493
Pesan hasil    : Sorting selesai: N=1000, median=5148
Exec time     : 0.000426 detik
Status exit   : 0 (WIFEXITED)
-----

Child index   : 1
PID anak      : 494
Pesan hasil    : Search selesai: target=1234 TIDAK ditemukan (return -1)
Exec time     : 0.000068 detik
Status exit   : 0 (WIFEXITED)
-----

Child index   : 2
PID anak      : 495
Pesan hasil    : Calculation selesai: primes <= 30000 : 3245
Exec time     : 0.000634 detik
Status exit   : 0 (WIFEXITED)
-----

Parent: semua anak telah dikumpulkan. Selesai.
```


TUGAS 2 CHAT APPLICATION DENGAN IPC

Implementasikan aplikasi chat sederhana menggunakan:

- Shared Memory atau Message Queue untuk menyimpan pesan
- Multiple processes sebagai pengguna
- Mutex/Semaphore untuk sinkronisasi
- Fitur: kirim pesan, lihat history, exit

TUJUAN SETIAP PERCOBAAN:

- Mengimplementasikan komunikasi antar proses menggunakan Shared Memory atau Message Queue.
- Mempelajari penggunaan mutex/semaphore untuk sinkronisasi akses memori bersama.
- Membuat aplikasi chat sederhana yang memungkinkan kirim pesan, lihat history, dan exit.

DASAR TEORI

1. Konsep Proses

- Proses adalah program yang sedang dieksekusi oleh sistem operasi.
- Proses dapat berkomunikasi menggunakan IPC (Inter-Process Communication).

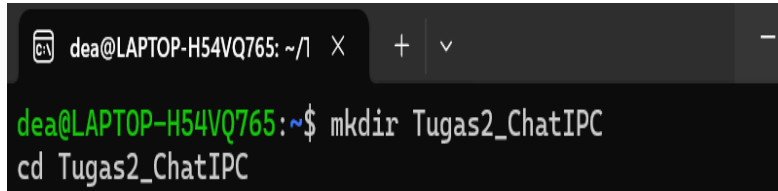
2. IPC Methods

- Shared Memory: Memori yang bisa diakses oleh beberapa proses. Cepat karena proses langsung membaca/menulis memori yang sama.
- Message Queue: Sistem antrian pesan antar proses, aman tapi lebih lambat dibanding shared memory.
- Semaphore/Mutex: Digunakan untuk sinkronisasi akses resource agar tidak terjadi race condition.

PERCOBAAN PRAKTIKUM CHAT APPLICATION DENGAN IPC

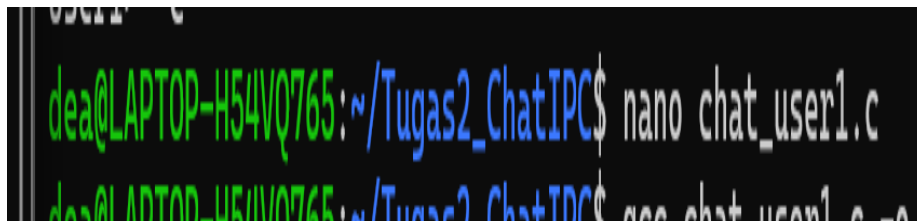
Membuat file source code

1. Buat folder proyek:

A terminal window with a dark background. The prompt is 'dea@LAPTOP-H54VQ765: ~/1'. The user enters 'mkdir Tugas2_ChatIPC' and then 'cd Tugas2_ChatIPC'.

```
dea@LAPTOP-H54VQ765: ~/1 X + v -
dea@LAPTOP-H54VQ765:~$ mkdir Tugas2_ChatIPC
dea@LAPTOP-H54VQ765:~$ cd Tugas2_ChatIPC
```

2. Buat di terminal 1 file chat_user1.c:

A terminal window with a dark background. The prompt is 'dea@LAPTOP-H54VQ765: ~/Tugas2_ChatIPC'. The user enters 'nano chat_user1.c'.

```
dea@LAPTOP-H54VQ765:~/Tugas2_ChatIPC$ nano chat_user1.c
```

KODE PROGRAM nano chat_user1.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/mman.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
#include <semaphore.h>
```

```
#define SHM_NAME "/chat_shm"
```

```
#define SEM_MUTEX "/chat_mutex"
```

```
#define BUFFER_SIZE 10
```

```
#define MSG_SIZE 256
```

```
typedef struct {  
    int head;  
    int tail;  
    int count;  
    char messages[BUFFER_SIZE][MSG_SIZE];  
    char users[BUFFER_SIZE][32];  
} chat_shm_t;
```

```
void print_history(chat_shm_t *shm) {  
    int idx = shm->head;  
    for(int i=0; i<shm->count; i++){  
        printf("%s> %s\n", shm->users[idx], shm->messages[idx]);  
        idx = (idx + 1) % BUFFER_SIZE;  
    }  
}
```

```
int main() {  
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);  
    ftruncate(shm_fd, sizeof(chat_shm_t));  
    chat_shm_t *shm = mmap(0, sizeof(chat_shm_t), PROT_READ | PROT_WRITE,  
MAP_SHARED, shm_fd, 0);  
  
    sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
```

```

// Inisialisasi shared memory

shm->head = 0;

shm->tail = 0;

shm->count = 0;


char msg[MSG_SIZE];

printf("=== CHAT ROOM ===\n");


while(1){

    printf("User1> ");

    fgets(msg, MSG_SIZE, stdin);

    msg[strcspn(msg, "\n")] = 0; // remove newline


    if(strcmp(msg, "/exit") == 0) break;


    sem_wait(mutex);

    strcpy(shm->messages[shm->tail], msg);

    strcpy(shm->users[shm->tail], "User1");

    shm->tail = (shm->tail + 1) % BUFFER_SIZE;

    if(shm->count < BUFFER_SIZE) shm->count++;

    else shm->head = (shm->head + 1) % BUFFER_SIZE;

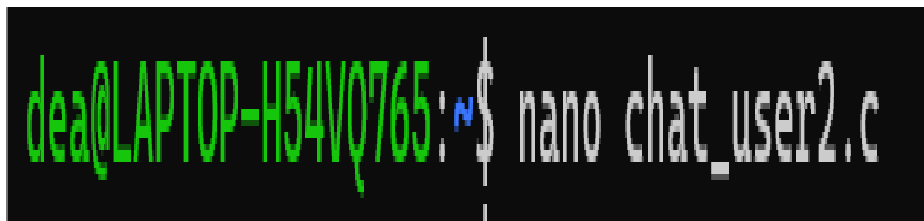
    sem_post(mutex);


    // Print history

```

```
    sem_wait(mutex);  
    print_history(shm);  
    sem_post(mutex);  
}  
  
munmap(shm, sizeof(chat_shm_t));  
close(shm_fd);  
return 0;  
}
```

3. Buat di terminal 2 file chat_user2.c:



KODE PROGRAM nano chat_user2.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/mman.h>  
#include <sys/stat.h>  
#include <fcntl.h>  
#include <semaphore.h>
```

```
#define SHM_NAME "/chat_shm"
```

```
#define SEM_MUTEX "/chat_mutex"
```

```
#define BUFFER_SIZE 10
```

```
#define MSG_SIZE 256
```

```
typedef struct {
```

```
    int head;
```

```
    int tail;
```

```
    int count;
```

```
    char messages[BUFFER_SIZE][MSG_SIZE];
```

```
    char users[BUFFER_SIZE][32];
```

```
} chat_shm_t;
```

```
void print_history(chat_shm_t *shm) {
```

```
    int idx = shm->head;
```

```
    for(int i=0; i<shm->count; i++){
```

```
        printf("%s> %s\n", shm->users[idx], shm->messages[idx]);
```

```
        idx = (idx + 1) % BUFFER_SIZE;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
```

```
    chat_shm_t *shm = mmap(0, sizeof(chat_shm_t), PROT_READ | PROT_WRITE,  
MAP_SHARED, shm_fd, 0);
```

```
sem_t *mutex = sem_open(SEM_MUTEX, 0);
```

```
char msg[MSG_SIZE];
```

```
printf("=== CHAT ROOM ===\n");
```

```
while(1){
```

```
    // Print history
```

```
    sem_wait(mutex);
```

```
    print_history(shm);
```

```
    sem_post(mutex);
```

```
    printf("User2> ");
```

```
    fgets(msg, MSG_SIZE, stdin);
```

```
    msg[strcspn(msg, "\n")] = 0; // remove newline
```

```
    if(strcmp(msg, "/exit") == 0) break;
```

```
    sem_wait(mutex);
```

```
    strcpy(shm->messages[shm->tail], msg);
```

```
    strcpy(shm->users[shm->tail], "User2");
```

```
    shm->tail = (shm->tail + 1) % BUFFER_SIZE;
```

```
    if(shm->count < BUFFER_SIZE) shm->count++;
```

```
    else shm->head = (shm->head + 1) % BUFFER_SIZE;
```

```
    sem_post(mutex);
```

```

}

munmap(shm, sizeof(chat_shm_t));

close(shm_fd);

return 0;
}

```

4. Compile dan jalankan di 2 terminal berbeda:

Terminal 1:

```

dea@LAPTOP-H54VQ765:~/Tugas2_ChatIPC$ gcc chat_user1.c -o chat_u
ser1 -lrt -pthread
dea@LAPTOP-H54VQ765:~/Tugas2_ChatIPC$ ./chat_user1
--- CHAT ROOM ---

```

Terminal 2:

```

dea@LAPTOP-H54VQ765:~$ gcc chat_user2.c -o chat_user2 -lrt -pthr
ead
dea@LAPTOP-H54VQ765:~$ ./chat_user2

```

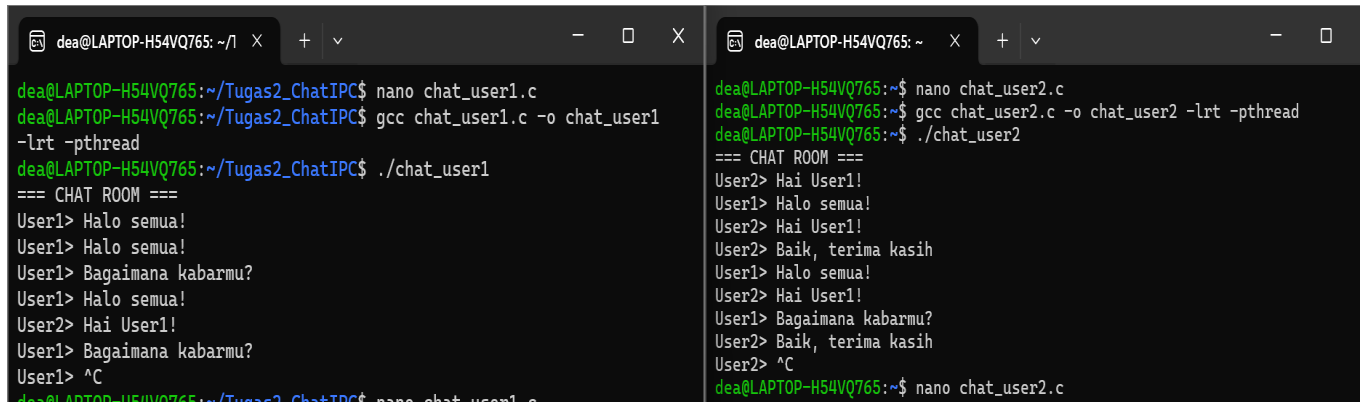
Penjelasan:

- -lrt → library real-time (untuk shared memory)
- -pthread → library untuk semaphore

5. Hasil Output

Terminal 1

Terminal 2



```
dea@LAPTOP-H54VQ765: ~/Tugas2_ChatIPC$ nano chat_user1.c
dea@LAPTOP-H54VQ765: ~/Tugas2_ChatIPC$ gcc chat_user1.c -o chat_user1 -lrt -pthread
dea@LAPTOP-H54VQ765: ~/Tugas2_ChatIPC$ ./chat_user1
=== CHAT ROOM ===
User1> Halo semua!
User1> Halo semua!
User1> Bagaimana kabarmu?
User1> Halo semua!
User2> Hai User1!
User1> Bagaimana kabarmu?
User1> ^C
dea@LAPTOP-H54VQ765: ~/Tugas2_ChatIPC$ nano chat_user1.c

dea@LAPTOP-H54VQ765: ~$ nano chat_user2.c
dea@LAPTOP-H54VQ765: ~$ gcc chat_user2.c -o chat_user2 -lrt -pthread
dea@LAPTOP-H54VQ765: ~$ ./chat_user2
=== CHAT ROOM ===
User2> Hai User1!
User1> Halo semua!
User2> Hai User1!
User2> Baik, terima kasih
User1> Halo semua!
User2> Hai User1!
User1> Bagaimana kabarmu?
User2> Baik, terima kasih
User2> ^C
dea@LAPTOP-H54VQ765: ~$ nano chat_user2.c
```

Penjelasan:

Terminal 1 menampilkan input User1 dan pesan User2.

Terminal 2 menampilkan input User2 dan pesan User1.

Analisis Hasil

1. Program berhasil membuat dua proses chat yang bisa saling bertukar pesan.
2. Shared memory digunakan untuk menyimpan pesan bersama sehingga kedua proses dapat membaca dan menulis ke buffer yang sama.
3. Semaphore memastikan hanya satu proses yang mengakses shared memory pada satu waktu, mencegah race condition.
4. Buffer bersifat circular, sehingga pesan lama akan tergantikan saat buffer penuh.

IMPLEMENTASI TUGAS 2

A. Shared Memory untuk menyimpan pesan

Kode Terminal 1:

```
GNU nano 7.2 chat_user1.c
int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
ftruncate(shm_fd, sizeof(chat_shm_t));
chat_shm_t *shm = mmap(0, sizeof(chat_shm_t), PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

Kode terminal2:

```
int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
chat_shm_t *shm = mmap(0, sizeof(chat_shm_t), PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

Penjelasan:

- shm_open membuat atau membuka shared memory dengan nama /chat_shm.
- O_CREAT | O_RDWR pada User1 → membuat memory baru jika belum ada.
- O_RDWR pada User2 → membuka memory yang sudah dibuat User1.
- ftruncate → mengatur ukuran shared memory sesuai struktur chat_shm_t.
- mmap → memetakan shared memory agar bisa dibaca/ditulis oleh proses.
- Fungsi shared memory: Menyimpan pesan (messages) dan nama user (users) sehingga kedua terminal bisa saling membaca dan menulis pesan.

B. Multiple Processes sebagai pengguna

Penjelasan:

- Terminal 1 → proses User1
- Terminal 2 → proses User2
- Kedua proses berjalan bersamaan, masing-masing mengakses shared memory yang sama.
- Dengan cara ini, kedua user dapat berkomunikasi secara real-time.

Contoh interaksi:

1. User1 mengetik “Halo semua!”
2. User2 membaca pesan tersebut dari shared memory dan membalas “Hai User1!”
3. User1 melihat balasan User2 langsung di terminalnya.

C. Mutex/Semaphore untuk sinkronisasi

Inisialisasi Terminal 1:

```
sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
```

Inisialisasi Terminal 2:

```
sem_t *mutex = sem_open(SEM_MUTEX, 0);
```

Penggunaan saat menulis pesan:

Terminal 1

```
sem_wait(mutex);  
strcpy(shm->messages[shm->tail], msg);  
strcpy(shm->users[shm->tail], "User1");  
shm->tail = (shm->tail + 1) % BUFFER_SIZE;  
if(shm->count < BUFFER_SIZE) shm->count++;  
else shm->head = (shm->head + 1) % BUFFER_SIZE;  
sem_post(mutex);
```

Terminal 2:

```
sem_wait(mutex);
strcpy(shm->messages[shm->tail], msg);
strcpy(shm->users[shm->tail], "User2");
shm->tail = (shm->tail + 1) % BUFFER_SIZE;
if(shm->count < BUFFER_SIZE) shm->count++;
else shm->head = (shm->head + 1) % BUFFER_SIZE;
sem_post(mutex);
}
```

Penggunaan saat membaca history:

```
// Print history
sem_wait(mutex);
print_history(shm);
sem_post(mutex);
}
```

Penjelasan:

- `sem_wait` → proses menunggu giliran sebelum masuk **critical section** (akses shared memory).
- `sem_post` → selesai akses, semaphore dilepas agar proses lain bisa masuk.
- Dengan semaphore, race condition dihindari saat kedua terminal menulis/menampilkan pesan bersamaan.

D. Fitur Chat: kirim pesan, lihat history, exit

Fitur	Implementasi dalam kode	Penjelasan
Kirim pesan	Menulis ke <code>shm->messages</code> dan <code>shm->users</code>	Pesan yang diketik user disimpan ke buffer circular, sehingga bisa dibaca proses lain.
Lihat history	<code>print_history(shm)</code>	Fungsi ini menampilkan semua pesan dari head sampai tail sesuai urutan input.
Exit	<code>if(strcmp(msg, "/exit") == 0) break;</code>	Perintah <code>/exit</code> mengakhiri loop, proses keluar dengan rapi.

KESIMPULAN PRAKTIKUM

1. Pembelajaran dari Praktikum

Tugas 1: Sistem Manajemen Proses

- Belajar membuat multiple child processes menggunakan `fork()`.
- Setiap proses dapat melakukan tugas berbeda seperti sorting, searching, dan perhitungan.
- Belajar mengukur execution time setiap proses menggunakan `gettimeofday()` atau `clock()`.
- Memahami cara parent process mengkoordinasi dan mengumpulkan hasil dari child processes menggunakan `wait()` atau `waitpid()`.
- Mempelajari manajemen status proses, termasuk menangani child yang selesai dan zombie process.

Tugas 2: Chat Application dengan IPC

- Mempelajari Inter-Process Communication (IPC) menggunakan Shared Memory atau Message Queue.
- Menangani sinkronisasi akses resource dengan Semaphore/Mutex agar tidak terjadi race condition.
- Belajar membuat multi-process chat (dua terminal) dengan fitur: kirim pesan, lihat history, dan exit.
- Memahami buffer circular untuk menyimpan pesan agar penggunaan memory tetap stabil.
- Mengamati real-time data sharing antar proses melalui shared memory.

2. Perbandingan Metode IPC

Aspek	Tugas 1	Tugas 2
Metode IPC	Umumnya pipe atau shared memory untuk komunikasi hasil proses	Shared Memory + Semaphore (atau Message Queue)
Tujuan komunikasi	Mengirim hasil proses dari child ke parent	Menyimpan pesan chat agar bisa diakses banyak proses
Sinkronisasi	Tidak terlalu kompleks (parent menunggu child)	Penting → semaphore digunakan untuk menghindari race condition
Kelebihan	Mudah untuk mengelola child dan mengumpulkan hasil	Cepat, real-time, langsung akses memori
Kekurangan	Perlu koordinasi parent-child	Jika banyak user, semaphore bisa menjadi bottleneck
Skalabilitas	Terbatas pada jumlah child yang dapat dikelola	Efisien untuk beberapa user, tapi kurang scalable jika user sangat banyak